`

CASE STUDY REPORT

Opinion Mining and Tag-Based Categorization of IMDb Movie Reviews

Course: Text Mining

Student: Mangai Varsitha M

Faculty: Gina George

Date: 21/10/2026

Department / Registration Number: MID / 23MID0015

`

Source Code and Dataset Access

Source code (Github sharing link): https://github.com/mangaivarsitham2023-commits/TextMiningDA.git

Notebook file: IMDb_Opinion_Mining_Dashboard.ipynb

Dataset: UCI Sentiment Labelled Sentences, IMDb subset (imdb_labelled.txt).

Dataset page: https://archive.ics.uci.edu/dataset/331/sentiment+labelled+sentences

Abstract

This case study examines how short IMDb movie-review sentences can be organized using text mining and sentiment classification. A publicly available, manually labelled subset of 1,000 IMDb review sentences was selected from the UCI Sentiment Labelled Sentences collection. Four entries became empty after preprocessing, leaving 996 usable reviews. The processing stages included text cleaning, tokenization, stop-word removal with negation retention, stemming, Bag-of-Words, TF-IDF, and a two-component Truncated SVD visualization. A Logistic Regression classifier was trained using 796 reviews and tested on 200 held-out reviews. Its test accuracy was 83.00%, corresponding to 166 correct predictions. To summarize frequently discussed movie aspects, the notebook also extracted, refined, and categorized review tags. A Jupyter-based interactive dashboard brings together the tables, plots, model evaluation, tag cloud, and a sample prediction tool. The work shows how short text can be converted into structured evidence while recognizing the limits of a small, binary-labelled dataset.

1. Introduction

Movie reviews often contain direct opinions about performances, stories, dialogue, pacing and the overall viewing experience. When many comments are collected together, manually reading and grouping every review becomes difficult. Text mining can convert the words in those comments into measurable patterns. Opinion mining then helps classify each review according to its expressed sentiment.

This study uses a single IMDb dataset rather than combining reviews from different platforms. The results are produced in Python and presented in an interactive dashboard running inside Jupyter Notebook. The aim is not only to predict sentiment, but also to explain which words and movie-related topics appear in the review collection.

2. Problem Statement

A collection of short movie reviews is difficult to analyze consistently because it contains different writing styles, common words and varied ways of expressing praise or criticism. The problem addressed in this case study is to process the raw review sentences, represent them numerically, predict positive or negative sentiment, identify useful review tags, and display the outcomes through a single dashboard.

3. Objectives

- Examine the structure and class distribution of a single labelled IMDb review dataset.

`

- Clean and tokenize the reviews while retaining words that reverse sentiment, such as “not”.
- Represent text with Bag-of-Words and TF-IDF and compare frequent features.
- Use Truncated SVD to create an exploratory two-dimensional representation.
- Train and evaluate a supervised positive/negative sentiment classifier on an unseen test split.
- Extract and refine informative review tags and group recognized tags by theme.
- Present graphs, tables and model predictions through a Jupyter Notebook dashboard.

4. Dataset Description

The IMDb portion of the UCI Sentiment Labelled Sentences dataset was used. It contains 1,000 labelled movie-review sentences: 500 negative and 500 positive. The complete UCI collection also provides Amazon and Yelp examples, but those were not included in this case study. The IMDb data has two primary fields: the review sentence and the sentiment label (0 for negative; 1 for positive). The dataset does not provide a neutral ground-truth class.

Dataset characteristic	**Observed value**
Dataset / subset	UCI Sentiment Labelled Sentences — IMDb
Original records	1,000
Original class distribution	500 Negative; 500 Positive
Missing values in loaded fields	0
Duplicate rows observed	3
Rows empty after cleaning	4
Final records used	996
Final class distribution	500 Negative; 496 Positive
Label values	0 = Negative; 1 = Positive

The source notebook reported three duplicate rows during exploration. The implemented cleaning stage removed only reviews that became empty; duplicates were not separately deleted. Consequently, the analysis reflects the actual executed notebook, rather than a revised dataset.

5. Methodology

The workflow followed this order: load IMDb review sentences → inspect data → clean text → build Bag-of-Words and TF-IDF features → reduce dimensions for visualization → train and test Logistic Regression → generate descriptive tags → export results → display the dashboard.

5.1 Dataset Loading and Examination

The dataset was read into a pandas DataFrame. The notebook printed the record count, missing-value counts and duplicate count, and displayed the first ten original sentences and labels. This step confirmed that the source contained 1,000 labelled IMDb entries.

`

5.2 Text Cleaning and Preprocessing

Each sentence was converted to lowercase and tokenized using alphabetic words. The program filtered common English stop words, retained negation terms such as “not”, and applied the Snowball English stemmer to reduce related word forms. Short or empty processed texts were checked. Four records became empty after this stage and were excluded. For example, a word such as “moving” may be reduced to “move”; stemming can also produce forms such as “movi”.

Unlike lemmatization, stemming does not always produce a dictionary word. This is expected. The cleaned column was used for vectorization, while the original review sentences remained available for inspection and readable tag extraction.

5.3 Bag-of-Words (BoW)

CountVectorizer transformed the processed text into a review-by-word count matrix. The resulting BoW matrix contained 996 rows and 2,000 columns. Counts were summed across the corpus to identify frequently occurring stems. Among the most frequent were “movi” (211), “film” (189), “not” (177), “bad” (73), and “good” (58). These frequency values describe how often features appear, not whether they are positive or negative.

	term	frequency
0	movi	211
1	film	189
2	not	177
3	bad	73
4	it	69
5	just	63
6	like	60
7	charact	59
8	good	58
9	time	50
10	watch	48
11	act	48
12	great	42
13	realli	41
14	love	37
15	make	36
16	scene	33
17	did	33
18	stori	32
19	no	31

Description:

The table shows the 20 most frequently occurring terms in the IMDb reviews after text preprocessing. Words such as movi (211), film (189), and bad (73) appear most often. This helps identify the common words used by reviewers and provides a basic understanding of the review content before sentiment classification.

`

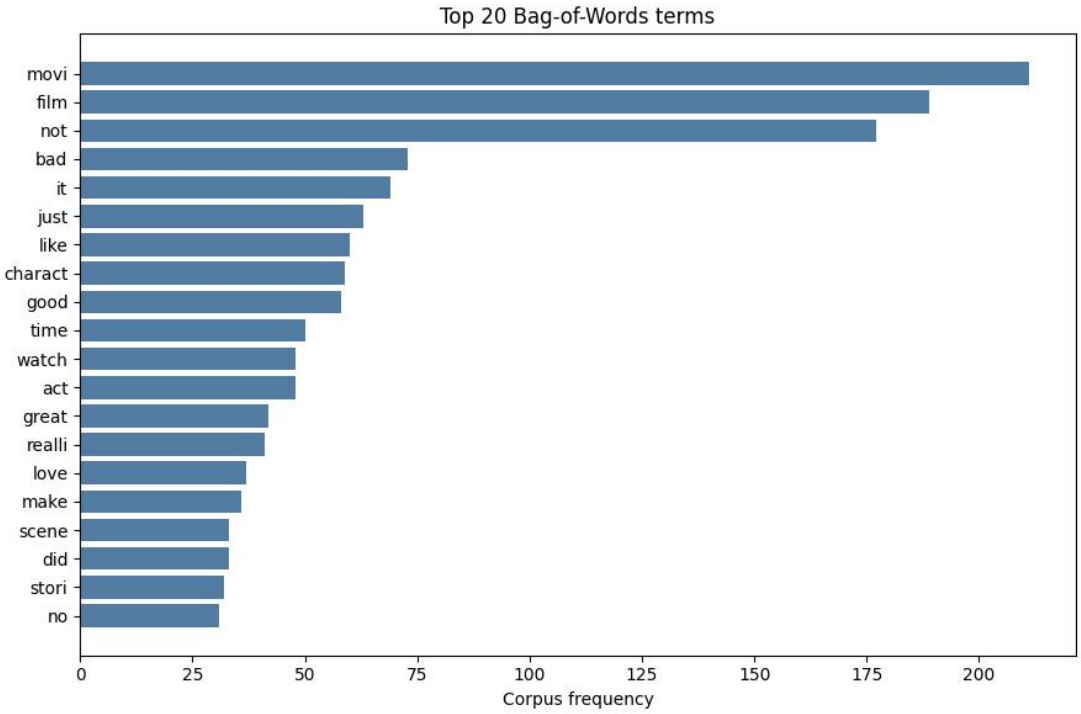


Figure 1. Top Bag-of-Words features from the IMDb reviews.

5.4 TF-IDF Feature Extraction

TF-IDF assigns numerical weights to words and two-word phrases. Here, TfidfVectorizer used unigram and bigram features, a maximum of 2,000 features, and sublinear term frequency scaling for the descriptive analysis. The descriptive feature matrix had 996 rows and 2,000 columns. The highest mean weights included “movi” (0.038913), “film” (0.036377), “not” (0.031405), and “bad” (0.020557). Mean TF-IDF weight is not the same as a raw frequency count.

	term	mean_tfidf
0	movi	0.038913
1	film	0.036377
2	not	0.031405
3	bad	0.020557
4	just	0.017536
5	it	0.015959
6	good	0.015827
7	watch	0.014757
8	like	0.014032
9	charact	0.012929
10	act	0.012837
11	time	0.012752
12	realli	0.012640
13	great	0.012316
14	love	0.011470
15	scene	0.009349
16	stori	0.009081
17	best	0.009078
18	look	0.009052
19	no	0.009042

Description:

`

The table shows the top 20 terms ranked by their average TF-IDF scores in the IMDb reviews. Words such as movi (0.038913), film (0.036377), and not (0.031405) have the highest scores. These values help identify prominent terms in the reviews and provide useful numerical features for sentiment classification.

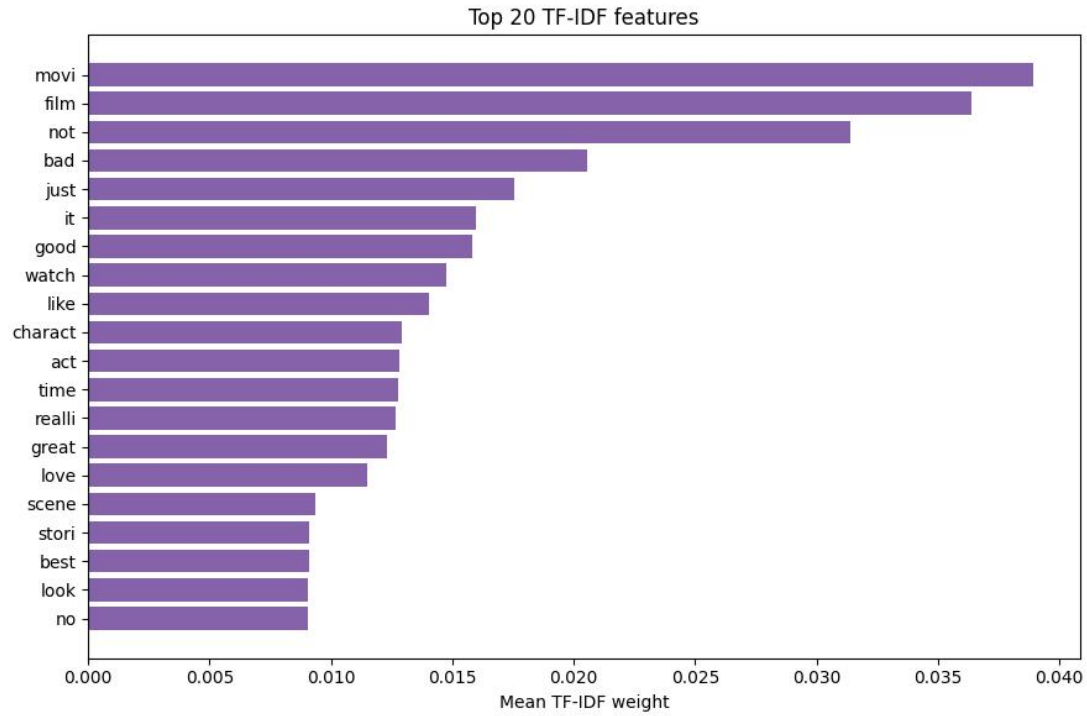
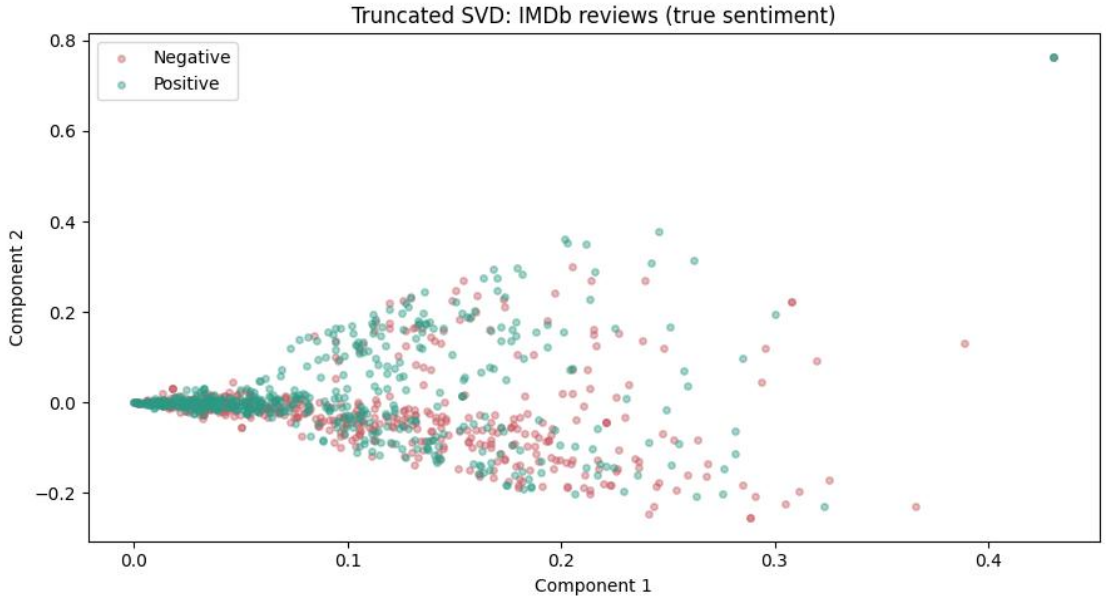


Figure 2. Ranked features based on mean TF-IDF weight.

5.5 Dimensionality Reduction with Truncated SVD

The high-dimensional descriptive TF-IDF matrix was projected onto two components using TruncatedSVD (random_state=42). Its shape changed from 996 × 2,000 to 996 × 2. The first two components together explained approximately 1.64% of the variation measured by the model. This small fraction means the scatter plot is useful for exploratory viewing, but it should not be treated as evidence that the two sentiment classes form distinct clusters.



`

Figure 3. Two-dimensional Truncated SVD/LSA projection of reviews.

Description:

The scatter plot shows IMDb reviews reduced to two dimensions using Truncated SVD. Green points represent positive reviews, while red points represent negative reviews. Most points overlap, showing that positive and negative reviews share similar text features. The plot helps visualize the review distribution, but the two sentiment classes are not clearly separated.

5.6 Supervised Sentiment Classification

The usable reviews were split into training and test subsets using an 80:20 split, stratified by sentiment label and with random_state=42. The supervised TF-IDF vectorizer was fitted on the training text only and then used to transform the unseen test reviews, avoiding train/test information leakage in the classification evaluation. Logistic Regression was then trained on the labelled training vectors.

The resulting training subset contained 796 reviews, while the held-out test subset contained 200 reviews. The model predicts one of two classes: Negative or Positive. Neutral was not included because no neutral class was available in the selected dataset.

5.7 Tag Generation, Ranking and Enrichment

A separate TF-IDF vectorizer analyzed the original, readable review text using unigrams and bigrams. It produced 1,170 candidate features. Generic terms and some unsuitable tokens were filtered out, leaving 1,072 refined candidates. The top 40 were displayed as ranked tags, based on mean TF-IDF weight. A rule-based set of movie-related keywords was used to assign themes such as Story & Plot, Acting & Characters, Direction & Production, Sound & Music, and Audience Experience. Terms that did not match a rule remained under “General opinion / other”.

This method offers a descriptive overview rather than an exact count of movie aspects mentioned by reviewers. Some useful terms may remain in the general category because the rule list is intentionally limited.

`

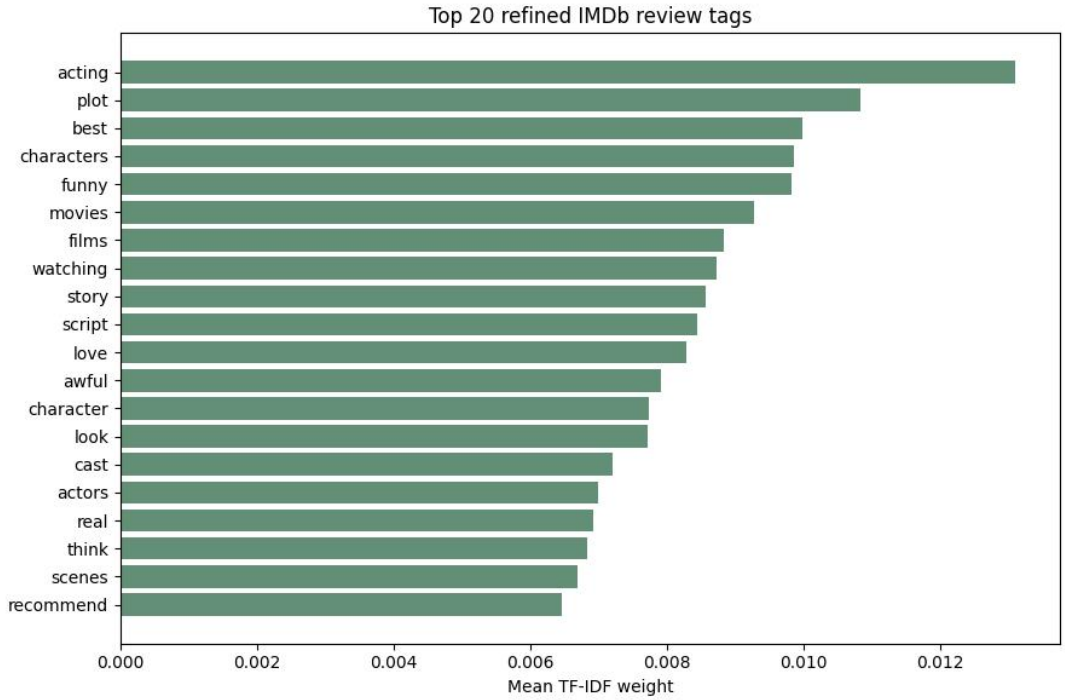
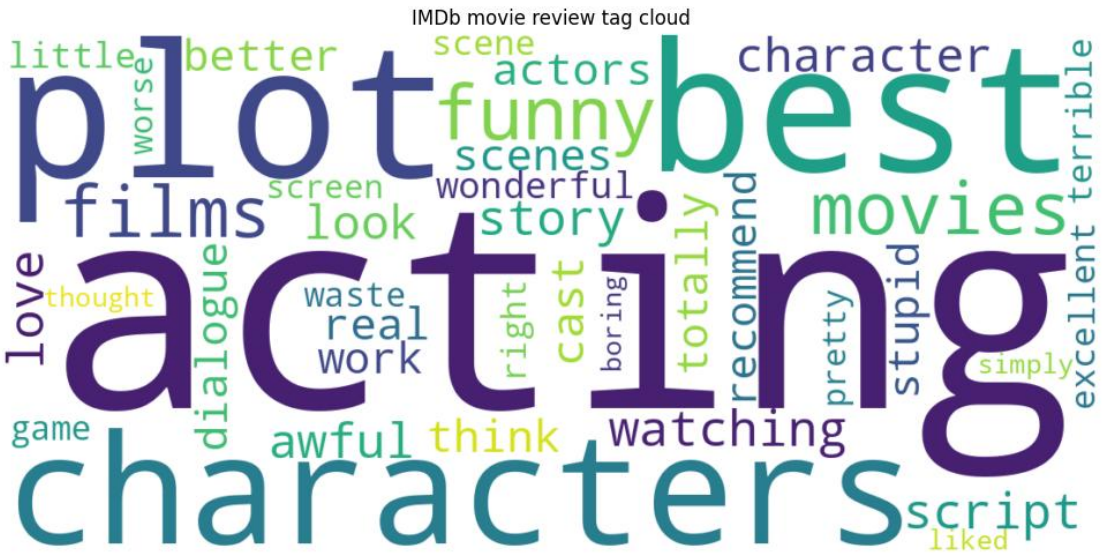


Figure 4. Mean TF-IDF ranking of refined review tags.

Description:

The bar chart shows the top 20 refined tags extracted from IMDb movie reviews using TF-IDF scores. Tags such as acting, plot, best, characters, and funny have higher scores. These tags highlight the main aspects discussed by reviewers, including acting performance, storyline, characters, and overall movie experience.



`

Figure 5. Word cloud generated using the selected review tags.

Description:

The word cloud displays the important tags extracted from IMDb movie reviews. Words such as acting, plot, best, and characters appear larger because they have higher TF-IDF weights. Other terms like funny, story, awful, and script highlight different aspects of the movies discussed by reviewers. This visualization helps identify the common topics and opinions expressed in the reviews.

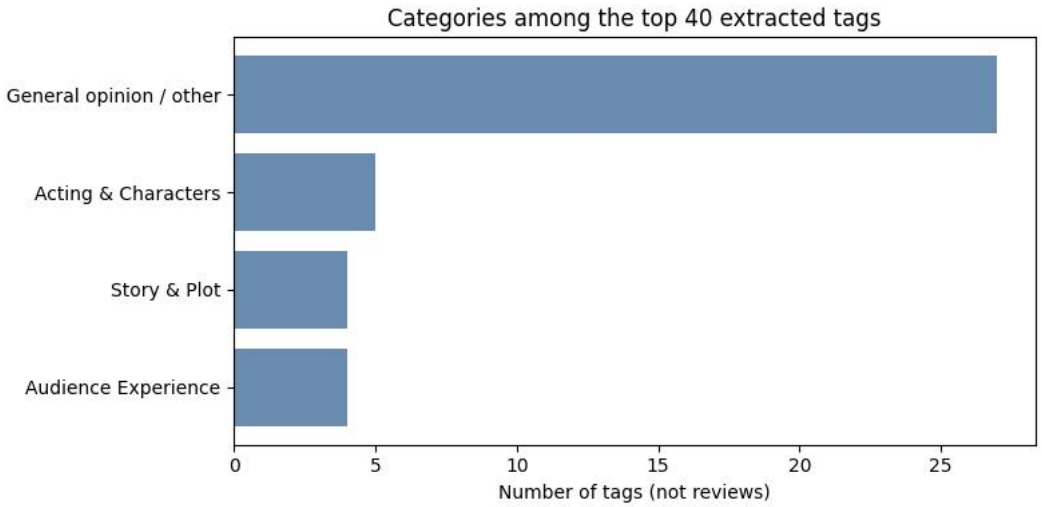


Figure 6. Number of selected tags assigned to each category.

Description:

The bar chart shows how the top 40 extracted IMDb review tags are grouped into different categories. Most tags belong to General Opinion / Other (27 tags), followed by Acting & Characters (5 tags), Story & Plot (4 tags), and Audience Experience (4 tags). This categorization helps organize the extracted words into meaningful groups and understand the main topics discussed in movie reviews.

6. Results and Analysis

6.1 Sentiment Distribution

The original dataset was evenly divided between positive and negative examples. After preprocessing, 996 reviews remained: 500 Negative and 496 Positive. The output graph provides a quick view of the nearly balanced class distribution of the retained data.

`

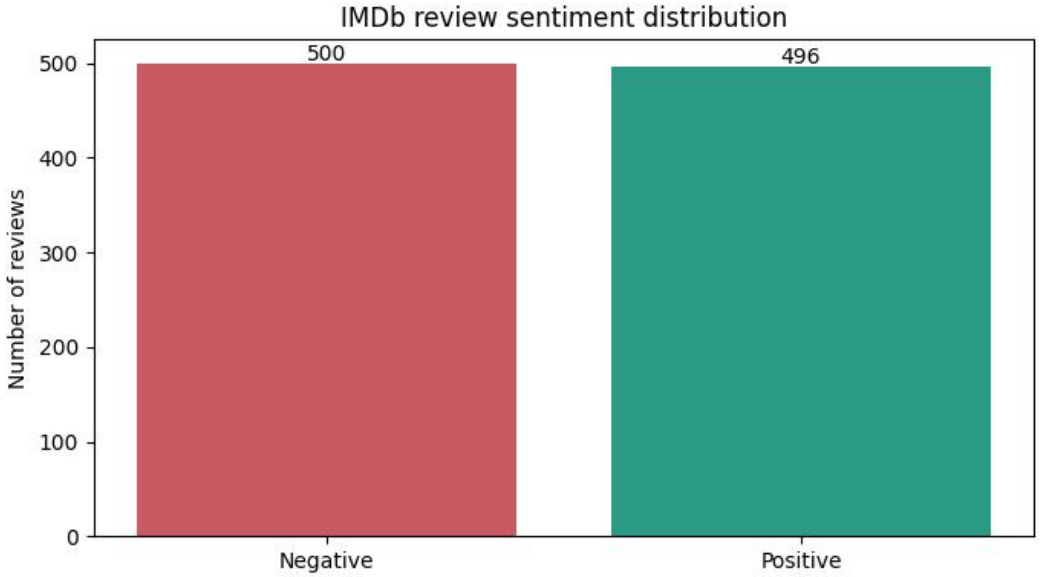


Figure 7. Sentiment distribution after preprocessing (996 usable reviews).

6.2 Classification Performance

The Logistic Regression classifier achieved 83.00% accuracy on the held-out test set. Among the 200 test examples, 166 were correctly classified and 34 were classified incorrectly. The model performed similarly on the positive and negative classes, with an F1-score of approximately 0.83 for each class.

Class	Precision	Recall	F1-score	Test samples
Negative	0.82	0.84	0.83	100
Positive	0.84	0.82	0.83	100
Macro average	0.83	0.83	0.83	200
Weighted average	0.83	0.83	0.83	200

The confusion matrix indicates that 84 negative reviews and 82 positive reviews were classified correctly. Sixteen actual negative reviews were predicted as positive, while 18 actual positive reviews were predicted as negative. The mistakes indicate that some short reviews contain wording that the classifier cannot distinguish reliably.

```
Training size: 796 Test size: 200
Test accuracy: 83.00%
              precision    recall  f1-score   support

    Negative       0.82      0.84      0.83       100
    Positive       0.84      0.82      0.83       100

    accuracy                           0.83       200
   macro avg       0.83      0.83      0.83       200
weighted avg       0.83      0.83      0.83       200
```

Description:

`

The confusion matrix shows the performance of the Logistic Regression model on 200 test reviews. The model correctly classified 84 negative reviews and 82 positive reviews. However, 16 negative reviews were predicted as positive, while 18 positive reviews were predicted as negative. Overall, the model correctly classified 166 out of 200 reviews, achieving an accuracy of 83%.

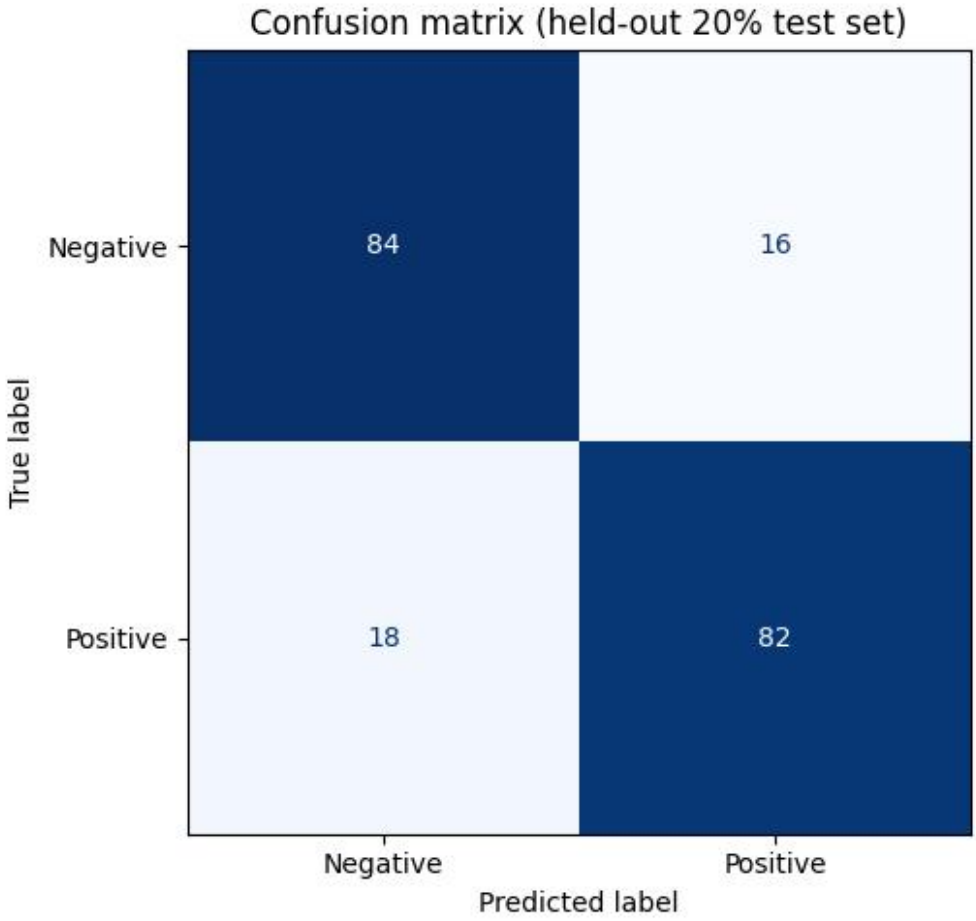


Figure 8. Confusion matrix for the held-out 200-review test set.

6.3 Tag Findings

The highest-ranked selected tags included “acting” (0.013102), “plot” (0.010832), “best” (0.009986), “characters” (0.009856), “funny” (0.009821), and “story” (0.008564). These tags suggest that the sample frequently contains discussion of performances, characters, story and audience response. Tag values are mean TF-IDF weights and should not be interpreted as numbers of people making a particular comment.

Tag	**Mean TF-IDF**	**Assigned group**
acting	0.013102	Acting & Characters
plot	0.010832	Story & Plot
characters	0.009856	Acting & Characters
funny	0.009821	Audience Experience

`

story	0.008564	Story & Plot
script	0.008444	Story & Plot
cast	0.007204	Acting & Characters

7. Dashboard Design and Output

An interactive dashboard was implemented inside Jupyter Notebook using ipywidgets rather than a separate hosted web application. It includes eight tabs: Overview, Preprocessing, Bag-of-Words, TF-IDF, SVD / LSA, Sentiment Evaluation, Tags & Categories, and Live Prediction. The tabs display the already generated plots and tables, allowing the student to inspect each stage from a single interface.

The Overview section shows summary counts and test accuracy alongside the review distribution and sample rows. The text-processing and vectorization tabs display examples, matrix dimensions, ranked features and corresponding graphs. The evaluation tab contains accuracy information, the classification report and confusion matrix. The tag tab shows refined tags, their categories and a word cloud. Finally, Live Prediction accepts typed movie-review text and produces a model-estimated Positive or Negative label with a positive-class probability.

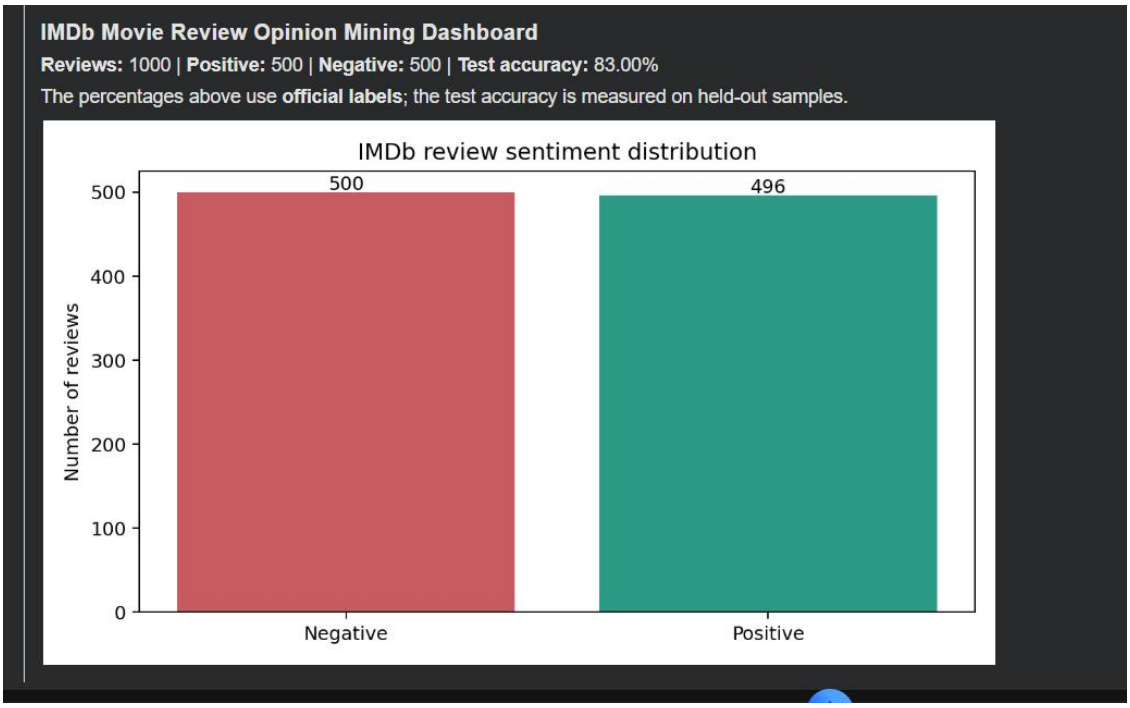


Figure 9. Screenshot of the interactive dashboard captured during notebook execution.

The dashboard contains predictions for the full processed corpus as well as separate test-set evaluation. Predictions on training reviews are not independent performance measurements; only the held-out subset is used to report the 83.00% accuracy.

8. Discussion

The results show that conventional text-mining methods can be useful for a compact sentiment dataset. BoW helped identify recurring vocabulary, while TF-IDF gave a weighted view of terms and phrases. The two-dimensional SVD plot

`

offered a visual summary of the text space, although its 1.64% explained variance restricts interpretation. Logistic Regression classified most unseen reviews correctly, but still confused 34 of 200 test cases. The tag analysis added a topic-oriented view by grouping some high-weight expressions into movie-related themes.

The dataset is small and contains short sentences rather than full movie reviews. It has only clearly positive and negative labels, so mixed or neutral opinions are not evaluated here. Stemming can make words less readable, and rule-based tag categorization can miss synonyms or ambiguous meanings. These limitations should be kept in mind when using the results outside this example dataset.

9. Conclusion

This case study completed the required text-mining stages on one IMDb review dataset. The implemented process cleaned the sentences, extracted Bag-of-Words and TF-IDF features, used SVD for visualization, and trained a supervised sentiment classifier. The Logistic Regression model obtained 83.00% accuracy on a held-out test set. Additional ranked tags, category summaries and an interactive Jupyter dashboard helped present the findings clearly. Overall, the work demonstrates how a simple Python pipeline can turn short movie-review comments into measurable text features and understandable visual outputs.

10. References

[1] D. Kotzias, “Sentiment Labelled Sentences,” UCI Machine Learning Repository, 2015. DOI: 10.24432/C57604. https://archive.ics.uci.edu/dataset/331/sentiment+labelled+sentences

[2] D. Kotzias, M. Denil, N. de Freitas, and P. Smyth, “From Group to Individual Labels Using Deep Features,” Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2015.

[3] Scikit-learn developers, “Scikit-learn User Guide,” https://scikit-learn.org/stable/user_guide.html (accessed 21 October 2026).

[4] Project implementation: IMDb_Opinion_Mining_Dashboard.ipynb, executed Jupyter Notebook and saved output figures, 21 October 2026.

Appendix A. Output Files and Evidence

The executed notebook records eight generated PNG graphs, four CSV tables and a metrics.json summary inside its output directory. Those files can be submitted with the notebook to support the findings in this report. The complete source-code sharing link must be added to the front section before submission.